

RELAZIONE PROGETTO LABORATORIO III

Studente: Francesco Lioni,

Matricola: 658189

1. Scelte Progettuali e Interpretazioni Personali

Alcune meccaniche di gioco sono state modellate secondo interpretazioni personali per ottimizzare l'esperienza utente:

- **Aggiornamento Differito delle Classifiche:** Le statistiche globali (Win Rate, Streak, ecc.) vengono ricalcolate e aggiornate *esclusivamente* al termine di ogni partita. Un utente appena registrato non figurerà nella classifica generale finché non avrà concluso il suo primo match.
- **Partecipazione "Opt-in":** Per non penalizzare gli utenti inattivi (AFK), l'inserimento nelle partite non è automatico. Un giocatore entra in partita al momento del login, ma per i match successivi vi partecipa solo se invia almeno una proposta (`submitProposal`).

2. Architettura Lato Server:

Il server adotta un'architettura multithread concorrente basata su tre componenti principali, progettata per separare il livello di rete dalla logica di gestione del gioco.

1.1 Gestione dei Thread (ServerMain e ServerWorker)

Il punto di ingresso è il `ServerMain`, che inizializza le strutture dati dai file di configurazione JSON e apre un `ServerSocketChannel` per gestire l'I/O in modalità NIO. All'arrivo di una nuova connessione TCP, il canale viene delegato a un task `ServerWorker`.

Per soddisfare il requisito della concorrenza è stato utilizzato un `CachedThreadPool(Executors.newCachedThreadPool())`. Questa scelta garantisce scalabilità automatica: crea nuovi thread per gestire i client in ingresso e riutilizza quelli inattivi, limitando lo spreco di risorse.

Ogni `ServerWorker` opera in un ciclo continuo: legge messaggi tramite `ByteBuffer`, formatta la stringa JSON in un oggetto polimorfico `Request`, invoca la logica e restituisce la `Response`. Ogni worker mantiene al suo interno lo stato locale della sessione (`loggedUser`), garantendo sicurezza sulle operazioni dell'utente senza dover reinviare le credenziali a ogni richiesta.

1.2 Il ServerController

L'intera logica di gestione della partita e la persistenza dei dati sono centralizzate nel `ServerController`, un'unica istanza passata per riferimento a tutti i worker. Gestisce le seguenti strutture dati critiche:

- `utentiOnline`: Una `ConcurrentHashMap<String, Boolean>` per tracciare i giocatori attivi e prevenire login multipli simultanei dello stesso account.

- **partitaCorrente:** Un oggetto `Partita` che racchiude lo stato in tempo reale (parole rimanenti, errori, proposte di tutti i giocatori attivi).
- **GameHistoryManager e RankingsManager:** Classi delegate alla storicizzazione asincrona su file JSON dei profili utente e delle classifiche globali a fine partita.

1.3 Primitive di Sincronizzazione e Timer di Gioco

Essendo il `ServerController` acceduto simultaneamente dai vari `ServerWorker`, è essenziale prevenire le *race condition*. L'accesso sicuro è garantito tramite due livelli di sincronizzazione:

1. **Concurrent Collections:** L'utilizzo esclusivo di `ConcurrentHashMap` rende thread-safe e atomiche le operazioni sulle mappe senza necessità di lock espliciti.
2. **Blocchi e Metodi Sincronizzati:** Tutti i metodi esposti dal controller (es. `handleSubmitProposalRequest`) sono dichiarati `synchronized`. Questo assicura che valutazioni complesse – come il controllo di una proposta di parole o l'assegnazione dei punteggi – vengano eseguite in mutua esclusione, impedendo che due client si vedano accreditato lo stesso gruppo nello stesso istante.
3. Infine, il controller avvia un thread indipendente (`GameTimerManager`) tramite un `SingleThreadExecutor`. Questo scandisce la durata del gioco (tramite `Thread.sleep`); allo scadere del timer, acquisisce il lock, chiude la partita, aggiorna le statistiche permanenti e innesca il broadcast UDP verso tutti i client con la classifica finale, per poi generare immediatamente la partita successiva.

1.4 Gestione delle Notifiche Asincrone (UDP)

la comunicazione degli eventi asincroni dal server ai client (nello specifico, l'avviso di fine partita) avviene separatamente tramite protocollo UDP. Questo compito è delegato alla classe `UdpNotificationManager`.

Per sapere a chi inviare i datagrammi, la classe utilizza una

`ConcurrentHashMap<Socket, InetAddress>`. Questa struttura associa il socket TCP della connessione principale del client (usato come chiave univoca) al suo indirizzo UDP (l'IP viene ricavato in modo sicuro dal socket TCP, mentre la porta è comunicata dal client in fase di connessione).

Allo scadere del timer di gioco, il controller invoca il metodo

`broadcastNotification()`, che itera in modo thread-safe sulla mappa e invia a ciascun giocatore un `DatagramPacket` contenente la classifica finale in formato JSON. L'uso della collezione concorrente garantisce che non si verifichino errori, se un giocatore si disconnette o viene rimosso esattamente durante la fase di invio del broadcast.

1.5 Persistenza dello Stato e File di Salvataggio

Per garantire che i dati non vadano persi al riavvio, il server implementa gestori dedicati (`UserManager`, `GameHistoryManager`, `RankingsManager`). La persistenza è

ottenuta serializzando le strutture in file JSON tramite la libreria `Gson`. All'avvio del sistema, i manager deserializzano il contenuto ripristinando lo stato in memoria. Ogni volta che si verifica un evento rilevante, la risorsa in memoria viene aggiornata e viene forzata una sovrascrittura del file JSON. Per scongiurare la corruzione dei file fisici a causa di accessi concorrenti, i metodi di I/O (come `saveRankings()`) sono protetti dalla primitiva `synchronized`, che funge da lock esclusivo.

1.6 Streaming del Dizionario delle Partite (GameProvider)

Per l'estrazione delle nuove partite dal dizionario JSON (`ConnectionsData`), si è scelto di *non* deserializzare l'intero file in memoria, poiché dizionari molto grandi potrebbero saturare la memoria del server. È stato invece implementato il `GameProvider`, che utilizza la classe `JsonReader` di `Gson` per effettuare una lettura in streaming sequenziale. Ad ogni avvio del timer, il provider consuma solo i byte necessari a estrarre il prossimo oggetto `Game`. Raggiunta la fine del file, il reader viene resettato, garantendo un ciclo infinito di partite a costo di memoria minimo.

3. Architettura Lato Client:

L'architettura del client è progettata separando la logica di I/O interattiva (sincrona) dalla ricezione passiva degli eventi di gioco (asincrona).

2.1 Gestione dei Thread e Flusso Principale (ClientMain)

Il client deve mantenere attivi due flussi di comunicazione paralleli. Per questo motivo, l'applicazione si avvale di soli due thread:

1. **Main Thread** : Apre un `SocketChannel` in modalità NIO verso il server e avvia in un ciclo infinito l'interfaccia a riga di comando. Tramite uno `Scanner`, legge i comandi dell'utente, delega alla classe `GameInput` la costruzione della richiesta JSON e la invia. Resta poi in attesa bloccante per leggere la risposta del server e inoltrarla a `GameUI` per il rendering grafico.
2. **Thread Asincrono UDP (UdpListener)**: Al boot, il client apre un `DatagramSocket` e lancia immediatamente un thread dedicato, gestito da un `SingleThreadExecutor`. Questo thread resta perennemente in ascolto passivo in background.

2.2 Comunicazione Asincrona e Sincronizzazione (UdpListener)

L'`UdpListener` ha il solo compito di intercettare il pacchetto broadcast inviato dal server allo scadere del timer di una partita.

Per evitare il problema della perdita di dati dovuta al limite dei datagrammi, il buffer di ricezione è stato impostato alla massima capienza teorica del payload UDP (65507 byte).

Trattandosi di due thread distinti (il Main che gestisce il login/logout e il Listener che stampa le classifiche), era necessario sincronizzarne lo stato. Ciò è stato implementato tramite il modificatore `volatile` associato alle variabili `running` e

`isLogged` nell'`UdpListener`. Questa primitiva garantisce che una modifica di stato effettuata dal Main (es. il giocatore effettua il logout) diventi immediatamente visibile al thread in ascolto, il quale scarterà automaticamente i broadcast successivi se l'utente non risulta più autenticato.

2.3 Astrazione della rete NIO (`NioUtils`)

Per soddisfare il requisito della comunicazione TCP tramite NIO, è stata implementata un'apposita libreria di utility condivisa (`NioUtils`).

Il metodo `sendJson` formatta la stringa aggiungendo il terminatore di riga `\n` e la invia in pacchetti di byte utilizzando il metodo `write` del `SocketChannel`.

Simmetricamente, il metodo `readJson` istanzia un `ByteBuffer` e un `ByteArrayOutputStream`. Legge i byte grezzi direttamente dal canale uno alla volta e li accumula in memoria finché non incontra il carattere `\n`. Questo approccio previene la corruzione della codifica UTF-8, assicurando che i caratteri speciali (come la *u* accentata) vengano ricostruiti correttamente durante la conversione finale in `String`.

4. Protocollo di Comunicazione e Design Pattern

Per garantire una comunicazione standardizzata tra Client e Server, è stato definito un protocollo basato su messaggi JSON. Le classi relative ai messaggi sono isolate in un package condiviso (`protocol`), suddivise in due gerarchie polimorfiche distinte aventi come radici le classi astratte `Request` e `Response`.

Per gestire la conversione tra rete e memoria, è stato implementato il Factory Method Pattern tramite le classi `HandlerRequest` e `HandlerResponse`. Queste factory analizzano la stringa JSON in ingresso estraendo un campo chiave obbligatorio ("`operation`"). Tramite un costrutto *switch*, il parser determina a runtime quale specifica classe concreta istanziare (es. `LoginRequest`, `GameStatsResponse`), delegando poi a `Gson` la deserializzazione completa del payload. Questa scelta ha reso l'aggiunta di nuovi comandi di gioco estremamente scalabile: è sufficiente creare una nuova classe ereditata e aggiungere un `case` alla factory.

5. Manuale utente

Una volta avviato, il Client presenterà un'interfaccia a riga di comando (CLI) intuitiva. Il sistema gestisce due stati principali, mostrando menu differenti a seconda che l'utente sia autenticato o meno.

- basta digitare la cifra indicata per eseguire il comando corrispondente.
- ogni comando poi genera la UI specifica che guida l'utente, nelle azioni da compiere

Menu non loggato	Menu loggato
<pre> SELEZIONA AZIONE 0 - Esci 1 - Registrati 2 - Accedi 3 - Aggiorna Credenziali </pre>	<pre> SELEZIONA AZIONE 0 - Esci 1 - Aggiorna Credenziali 2 - Logout 3 - Nuova Proposta 4 - Stato/Esito Partita 5 - Statistiche Partita 6 - Classifica Globale 7 - Statistiche Personali </pre>

- **Chiusura Client:** Digitare 0 (Esci) permette la disconnessione sicura e chiude il terminale.

Nota sulle Notifiche Asincrone (UDP)

Poiché le notifiche di fine partita arrivano in modo asincrono, potrebbero sovrapporsi visivamente a ciò che l'utente sta digitando sul terminale. Il problema è puramente grafico e non compromette i dati, ma si consiglia di attendere la notifica prima di inserire nuovi comandi a ridosso della scadenza del timer. (con Stato/Esito Partita viene mostrato il tempo rimanente)

6. Istruzioni per la compilazione

Prima di avviare il sistema, posizionarsi con il terminale nella radice del progetto.

Creazione cartella e Compilazione Universale:

(Nota: per evitare errori di formattazione durante il copia-incolla dal PDF, assicurarsi di eseguire il seguente comando su una singola riga continua)

```
javac -d bin -cp "lib/*" src/client/*.java src/server/*.java
src/server/model/*.java src/protocol/**/*.*java src/utils/*.java
```

Avvio del Sistema (Mac / Linux):

- **Server:** `java -cp "bin:lib/*" server.ServerMain`
- **Client:** `java -cp "bin:lib/*" client.ClientMain`

Avvio del Sistema (Windows):

- **Server:** `java -cp "bin;lib/*" server.ServerMain`
- **Client:** `java -cp "bin;lib/*" client.ClientMain`

Avvio dei jar:

- **Server:** `java -jar Server.jar`
- **Client:** `java -jar Client.jar`

NB: La libreria esterna *Gson* è già inclusa nel progetto all'interno della cartella `lib`. Non è richiesta alcuna installazione aggiuntiva